
Maestro Deepening Recommendations

How to Make Every Feature Richer, More Addictive, and More Useful
Using Amazon's Customer Obsession and Apple's Simplicity Principles

Prepared for: The CTO

Scope: Review of 11 features built across this coding session

Principles applied: Amazon Customer Obsession + Working Backwards; Apple Human Interface Guidelines (clarity, deference, depth)

Goal: Transform each feature from functional to addictive

Table of Contents

Table of Contents	2
1. Executive Summary	4
The Three Deepening Vectors	4
2. The Daily Briefing — From Summary to Morning Ritual	5
Recommendation 2.1: Push Delivery, Not Pull	5
Recommendation 2.2: One-Tap Execution	5
Recommendation 2.3: The Briefing Learns	6
3. Write-Back — From Gated to Effortless Governance	7
Recommendation 3.1: Progressive Trust	7
Recommendation 3.2: Contextual Write-Back Suggestions	7
4. Semantic Ask — From Search to Conversation	8
Recommendation 4.1: Synthesized Answers, Not Result Lists	8
Recommendation 4.2: Follow-Up Questions	8
5. Explanations — From Causal Chain to Trust Builder	10
Recommendation 5.1: Inline Explanations Everywhere	10
Recommendation 5.2: Explanation as Trust Signal	10
6. Role-Specific Playbooks — From Template to Persona	11
Recommendation 6.1: Persona-Adaptive Drafts	11
Recommendation 6.2: Playbook learns from edits	11
7. Four-Level Unknowns — From Map to Action	12
Recommendation 7.1: Unknown to Action Pipeline	12

8. Commitment Tracker — From Detection to Accountability	13
Recommendation 8.1: Commitments in the Briefing	13
Recommendation 8.2: Commitment Health Score	13
9. Organizational Timeline — From Log to Story	14
Recommendation 9.1: Narrative Timeline	14
10. Verified Knowledge — From Field to Workflow	15
Recommendation 10.1: Verification Prompts in Context	15
Recommendation 10.2: Verification Expiry	15
11. Task Intelligence — From Extraction to Completion	16
Recommendation 11.1: Auto-Completion Detection	16
Recommendation 11.2: Task Dependencies	16
12. Conversational Curiosity — From Q&A; to Teaching	17
Recommendation 12.1: Customer-Initiated Teaching	17
13. Cross-Feature Synergy — The Living Product	18
13.1 The Morning Flywheel	18
13.2 The Trust Loop	18
13.3 The Compounding Engine	18
14. Priority Matrix — What to Build Next	19
15. Conclusion	20

1. Executive Summary

This document reviews the 11 features built during this coding session and recommends specific, actionable deepening strategies for each. The recommendations are grounded in two design philosophies that the product must embody to win the enterprise market: Amazon's Customer Obsession (start with the customer and work backwards) and Apple's simplicity principles (clarity, deference, depth).

The features reviewed span three layers: the cognitive constitution (19 invisible engines across V2 through V6), the V8 intelligence upgrades (explanations, four-level unknowns, conversational curiosity, semantic ask, verified knowledge, commitment tracking), and the daily-work layer (trust, timeline, briefings, tasks, write-back, playbooks). Each layer is functional today. The question this document answers is: how does each feature become not just functional, but addictive? How does Maestro become the first tool the CEO opens in the morning and the last they check at night?

The core thesis is simple: Maestro's features are currently organized around what the system can do. They must be reorganized around what the customer needs to do. Every recommendation in this document follows from that shift. The daily-work features must feel instantaneous and effortless (Apple). The intelligence features must compound silently so the customer notices the improvement without seeing the machinery (Amazon's flywheel). And the two layers must never be visible as separate things — the customer sees one living product.

The Three Deepening Vectors

Every recommendation in this document targets one of three vectors. These vectors are derived from the design principles document and represent the axes along which a feature can be made richer, more addictive, and more useful.

Vector	Amazon Principle	Apple Principle	What It Means
1. Zero-Friction Actions	Customer Obsession: remove every obstacle between the customer and what they need	Clarity: the interface should only show what is needed, the customer's tasks	Every action the customer takes should require few steps
2. Silent Compounding	Working Backwards: start from the desired outcome and work backwards	Deference: the content the system feeds the user should make the next one smarter	Each interaction should make the next one smarter
3. Radical Honesty	Earn Trust: be vocally self-critical; admit what you don't know	Depth: visible knowledge of detail rewards curiosity	Every insight should show the missing piece. Every gap is a learning opportunity

2. The Daily Briefing — From Summary to Morning Ritual

The drafted daily briefing (Daily Work #3) currently produces four drafted artifacts: an email for the top recommendation, an email for the top money loss, a doc for the top knowledge trap, and an email for the top CEO-only decision. Each artifact has a recipient, a subject, a body, and evidence citations. This is functional. But it is not yet a morning ritual.

The difference between a summary and a ritual is the difference between a dashboard and a newspaper. A dashboard is consulted when something is wrong. A newspaper is opened every morning because it contains something you did not know you needed. Amazon's Bond lesson is clear: the morning briefing is the product. It is push, not pull. It is one artifact, not a dashboard. The CEO should open Maestro (or their Slack or their email) and find the briefing already there, already synthesized, already actionable.

Recommendation 2.1: Push Delivery, Not Pull

Current state: The briefing is generated on-demand when the CEO navigates to TODAY and the API is called.

Deepened state: The briefing is delivered. At 7:00 AM local time, Maestro pushes the briefing to the CEO's preferred channel: Slack DM, email, or the TODAY surface (pre-rendered). The CEO does not open Maestro to find the briefing; the briefing finds the CEO. This is the Bond model: the Presidential Brief is delivered, not fetched. Amazon's principle: start with the customer's morning routine and work backwards to the technology.

Implementation: Add a daily cron job (or APScheduler task) that calls the briefing API at 7:00 AM and pushes the result to the configured channel. If Slack is connected, post a card with the top 3 items + 'Open in Maestro' link. If email is preferred, send a formatted HTML email. The TODAY surface should cache the pre-rendered briefing so it loads instantly when the CEO opens it. The briefing should never show a loading spinner — it should be already there, like a newspaper on the doorstep.

Recommendation 2.2: One-Tap Execution

Current state: Each drafted artifact has a body with evidence citations. The CEO reads the draft, then separately navigates to Gmail or Jira to act on it.

Deepened state: Each drafted artifact has a single button: 'Send via Gmail' (creates a draft in the CEO's Gmail, one click to review and send), 'Create Jira ticket' (creates the ticket with evidence links), or 'Post to Slack' (posts the message to the relevant channel). The CEO never leaves the briefing. The gap between reading and acting is one tap, not five navigations. This is Apple's clarity principle: the action is visible at the moment of decision, not buried in a menu.

Implementation: Wire the existing write-back system (Daily Work #4) into each drafted artifact. The 'Send via Gmail' button calls POST /api/oem/writeback with provider=gmail, action_type=create_draft, and the artifact's to/subject/body. The CEO approves, the draft is created, Gmail opens. Total time from reading the briefing to having a draft in Gmail: under 10 seconds. This is the addictiveness loop: read,

tap, done.

Recommendation 2.3: The Briefing Learns

Current state: The briefing is generated from the model state. It does not learn from the CEO's behavior.

Deepened state: The briefing tracks which items the CEO clicks, which drafts they edit, which they ignore. Over time, the briefing prioritizes items the CEO consistently engages with and deprioritizes items they skip. If the CEO always clicks on bottleneck items but never clicks on velocity drops, the briefing leads with bottlenecks. This is Amazon's flywheel: every interaction makes the next briefing better. The CEO feels Maestro learning their preferences without being asked to configure anything. Apple's deference principle: the system adapts silently.

Implementation: Add an `attention_signal` table that logs which briefing items the CEO clicks (the click handler already exists for drill-down). Feed these signals into the briefing ranking algorithm. After 2 weeks of data, the briefing should noticeably prioritize the CEO's preferred topics. After 90 days, the briefing should feel personalized in a way the CEO cannot articulate but immediately notices when it is wrong.

3. Write-Back — From Gated to Effortless Governance

The write-back system (Daily Work #4) currently implements a preview-approve flow: the CEO previews an action (create Jira ticket, draft Gmail, post Slack), then approves it for execution. This is the correct governance model. But the current flow requires two explicit steps (preview, then approve) for every action. Over time, this friction will cause the CEO to skip write-backs and fall back to manual actions — defeating the purpose.

Recommendation 3.1: Progressive Trust

Current state: Every write-back requires preview + approve. No write-back is ever auto-executed.

Deepened state: Introduce progressive trust levels. A new user starts at Trust Level 1 (every action requires preview + approve). After 10 successful executions with no rejection, the user is promoted to Trust Level 2 (low-risk actions like posting to a team Slack channel are auto-executed; high-risk actions like creating Jira tickets still require approval). After 50 successful executions, Trust Level 3 (all actions auto-execute with a 5-second undo window, like Gmail's undo send). This is the Nerve lesson: action-taking works, but you must earn the right to act. Amazon's principle: deliver results, but earn trust first.

Implementation: Add a `trust_score` to the user profile. Increment on successful execution, decrement on rejection or undo. The write-back service checks the trust score and decides whether to auto-execute or require approval. The UI shows the trust level as a subtle badge (Apple's progressive disclosure: visible but not intrusive). The CEO can always demote themselves back to Level 1 from settings.

Recommendation 3.2: Contextual Write-Back Suggestions

Current state: Write-back is triggered manually from the execution plan modal. The CEO must click 'Prepare', then choose an action type.

Deepened state: Maestro proactively suggests write-backs in context. When the CEO reads a recommendation about a bottleneck, Maestro shows a subtle 'Create Jira ticket for this?' button inline. When the CEO reads a briefing item about an unresolved contradiction, Maestro shows 'Draft a Slack message to the team?'. The suggestion is contextual — it appears at the moment of decision, not in a separate menu. This is Apple's depth principle: the system reveals the next action exactly when the customer is thinking about it. Amazon's principle: reduce the distance between insight and action to zero.

Implementation: Add a `'suggested_actions'` field to every recommendation, briefing item, and contradiction. The frontend renders these as subtle inline buttons. The backend computes the suggestion based on the item type: bottleneck = Jira ticket, contradiction = Slack message, knowledge trap = Confluence doc. The CEO clicks, the preview appears, they approve. Three taps total instead of five navigations.

4. Semantic Ask — From Search to Conversation

The Semantic Ask (Competitor Feature B) replaced the keyword matcher with character n-gram TF-IDF cosine similarity. This fixed the live bug where 'should we hire more engineers' matched a churn learning object because both contained 'engineers'. The semantic matcher captures morphological variants and weights by term rarity. This is a solid foundation. But it is still search, not conversation. The customer asks a question, gets a list of matching laws and learning objects, and has to synthesize the answer themselves.

Recommendation 4.1: Synthesized Answers, Not Result Lists

Current state: The /ask endpoint returns a list of relevant laws and learning objects with relevance scores. The answer is a bullet-point summary of the matches.

Deepened state: The /ask endpoint returns a synthesized natural-language answer that weaves the evidence into a coherent paragraph. Instead of 'Based on 2 relevant execution laws: L-0001: priya.m is a bottleneck... L-0002: carlos.r is a bottleneck...', the answer reads: 'Priya and Carlos are both gating items in the review queue. Priya has been a bottleneck in 3 observed instances, primarily in the payments domain. Carlos gates architecture reviews. The pattern is consistent across both: single-person approval gates create queues.' This is the Glean lesson: return answers, not links. Amazon's principle: the customer wants the answer, not the evidence. The evidence is available on demand (click 'Why?'), but the default is the synthesized answer.

Implementation: Add a `synthesize_answer()` method to the `DecisionEngine` that takes the ranked laws + learning objects and produces a 2-3 sentence paragraph. The synthesis is rule-based for the pilot (template per question type: 'who is the bottleneck' = list the people + their domains + evidence count). In production, this is where the LLM integration happens: pass the ranked evidence to the LLM with a prompt that says 'synthesize this evidence into a 3-sentence answer. cite the law codes. do not hallucinate.' The LLM produces the natural-language answer; the semantic matcher provides the evidence.

Recommendation 4.2: Follow-Up Questions

Current state: Each /ask call is independent. The customer asks a question, gets an answer, and must formulate a new question from scratch to dig deeper.

Deepened state: After each answer, Maestro suggests 2-3 follow-up questions based on the evidence it found. If the answer is about bottlenecks, the follow-ups are: 'What would happen if Priya left?', 'When have we seen this before?', 'What's the cost of this bottleneck?' The customer clicks a follow-up instead of typing. This turns search into conversation — the same pattern as the Conversational Curiosity engine (V8 #3), applied to Ask. Apple's principle: the system anticipates the next question. Amazon's principle: reduce the cognitive load of formulating queries.

Implementation: After ranking the evidence, generate follow-up questions by examining the entities in the top results. If the top law mentions 'priya.m', suggest 'What if priya.m left?' (routes to the imagination engine). If the top law has a high `evidence_count`, suggest 'When have we seen this before?' (routes to the recall engine). The follow-ups are displayed as clickable chips below the answer. Each click submits a new /ask

query. The conversation builds naturally.

5. Explanations — From Causal Chain to Trust Builder

The ExplanationEngine (V8 #1) produces a 3-7 step causal chain for 'why' questions. Each step has evidence_count, confidence, and source references. The chain is rendered as a visual sequence with numbered markers and confidence bars. This is a strong foundation. But the explanation currently stands alone — it is not woven into the daily workflow.

Recommendation 5.1: Inline Explanations Everywhere

Current state: Explanations are available only when the customer explicitly asks a 'why' question in the ASK surface.

Deepened state: Every recommendation, every briefing item, every playbook output has a 'Why?' link that opens an inline explanation. The customer does not navigate to ASK and type a question; they click 'Why?' on the item they are already looking at, and the explanation unfolds inline. This is the Glean lesson: answers appear in the same interface where the question arises. Apple's principle: progressive disclosure. The explanation is hidden until the customer asks for it, then it appears in place. Amazon's principle: the customer should never have to go looking for the 'why' — it should be one tap away from every insight.

Implementation: The 'Why?' link infrastructure already exists (it was built in V8 #1 for confidence displays). Extend it to every recommendation card, every briefing item, every playbook section. The 'Why?' link calls the /explain endpoint with a context-derived question (e.g., for a bottleneck recommendation, the question is 'Why is everything bottlenecked?'). The explanation renders inline as a collapsible chain. The customer reads it, understands the cause, and either acts or asks a follow-up.

Recommendation 5.2: Explanation as Trust Signal

Current state: The explanation is informational. It does not feed back into the model.

Deepened state: When the customer reads an explanation and then takes action (approves a write-back, verifies a law, marks a task done), that action is a trust signal. Maestro learns: 'the customer trusted this explanation enough to act on it.' Over time, explanations with high trust signals are prioritized; explanations the customer consistently ignores are shortened. This is the Amazon flywheel: every interaction makes the next explanation better. The customer feels Maestro understanding not just what they asked, but what they found convincing. This is the deepest form of customer obsession: the system learns what the customer trusts, not just what they ask.

6. Role-Specific Playbooks — From Template to Persona

The PlaybookEngine (Daily Work #6) formats evidence differently for sales, marketing, and product roles. Sales gets drafted outreach with talking points. Marketing gets a campaign ROI table. Product gets a PRD outline with drafted tickets. Each playbook is a thin formatting layer over the existing engines. This is the right architecture. But the playbooks currently produce generic output — the same drafted email regardless of who the sales rep is, the same PRD outline regardless of the product manager's style.

Recommendation 6.1: Persona-Adaptive Drafts

Current state: The drafted outreach email uses a fixed template: 'Hi {customer} team, I wanted to follow up...' The PRD outline uses fixed section headers.

Deepened state: Maestro learns each user's writing style from their sent emails, Slack messages, and Confluence docs. The drafted outreach email matches the sales rep's voice: if they write casually ('Hey team, quick check-in...'), the draft is casual. If they write formally ('Dear {customer}, I hope this message finds you well...'), the draft is formal. The PRD outline matches the PM's preferred structure: if they always start with 'Problem', the outline starts with 'Problem'. If they start with 'User Story', the outline starts with 'User Story'. This is Apple's deference principle: the content is the interface, and the content should feel like the customer's own. Amazon's principle: the draft should be so good the customer only needs to edit 10%, not 90%.

Implementation: For each user, maintain a `writing_style` profile computed from their historical signals (Slack messages, emails, docs). The profile captures: formality level, average sentence length, preferred greeting/sign-off, common phrases. The playbook engine uses this profile to adapt the draft template. This is rule-based for the pilot (3-4 style profiles: casual, formal, technical, executive). In production, the LLM takes the user's last 10 emails as few-shot examples and generates the draft in their voice.

Recommendation 6.2: Playbook learns from edits

Current state: The CEO/sales rep/PM edits the drafted email or PRD, then sends it. The edit is lost.

Deepened state: Every edit is captured as a feedback signal. If the sales rep always removes the 'talking points' section and adds a personal anecdote, the next draft omits talking points and includes a placeholder for an anecdote. If the PM always adds a 'Risks' section to the PRD, the next PRD outline includes 'Risks' by default. This is the Dust lesson: teams improve agents via usage patterns. The playbook is not a static template; it is a living draft that converges on the user's preferred output over 10-20 iterations. The customer feels the drafts getting better without being asked to configure anything.

7. Four-Level Unknowns — From Map to Action

The Four-Level Unknowns (V8 #2) classify every organizational area into Known, Known Unknown, Unknown Unknown, and Emerging Unknown. This is rendered in TODAY as a collapsible section with 4 levels, each with different visual treatment. This is scientifically rigorous and builds trust. But it currently stops at classification — it does not drive action.

Recommendation 7.1: Unknown to Action Pipeline

Current state: The unknowns are displayed. The customer reads them and... does nothing, because there is no suggested action.

Deepened state: Each Unknown Unknown has a 'Connect a provider' button that suggests which integration would fill the gap (e.g., 'Connect Jira to start measuring the QA domain'). Each Emerging Unknown has an 'Investigate' button that creates a task for the relevant team lead. Each Known Unknown has an 'Instrument' button that suggests a specific metric to track. The unknowns are not just a map; they are a to-do list for organizational self-knowledge. Apple's principle: every piece of information has a clear next action. Amazon's principle: insight without action is waste.

Implementation: For each unknown level, compute a suggested_action based on the domain and the available providers. If the unknown domain is 'qa' and Jira is not connected, suggest 'Connect Jira'. If the unknown is emerging in 'security', suggest 'Create a task for the security team lead'. The action buttons call the write-back system (Daily Work #4) to create Jira tasks or the connector system to suggest provider connections. The unknowns become the input to the daily-work layer — the two layers feed each other.

8. Commitment Tracker — From Detection to Accountability

The CommitmentTracker (Competitor Feature E) scans signal text for commitment patterns ('I'll get back to you', 'will follow up by') and tracks their status: open, kept, broken. This is Bond's core feature on Maestro's evidence graph. But the tracker currently surfaces commitments only in a dedicated endpoint. The customer has to know to look for them.

Recommendation 8.1: Commitments in the Briefing

Current state: Commitments are available via GET /api/oem/commitments. The customer must navigate there explicitly.

Deepened state: The daily briefing includes a 'Commitments due today' section. Any commitment with a due_date of today or earlier appears in the morning briefing with the person who committed, the person they committed to, and the original context. The CEO sees 'Priya promised to get back to Raj by Friday about the OAuth review' and can nudge Priya with one tap ('Send a gentle reminder'). Broken commitments from the past week appear in a 'Broken this week' section. This makes the commitment tracker ambient — the customer does not look for commitments; commitments find them. The Bond lesson: the morning brief is the product.

Recommendation 8.2: Commitment Health Score

Current state: Each commitment is open, kept, or broken. There is no aggregate view.

Deepened state: Maestro computes a commitment health score per person: the ratio of kept to broken commitments over the last 90 days. This score appears in the knowledge graph next to each person's influence score. A person with high influence but low commitment health is a risk: they promise but do not deliver. A person with moderate influence but high commitment health is a reliable executor. This is the organizational DNA (V6) applied to commitments — the org's character is revealed not by what it says, but by what it follows through on. Amazon's principle: measure what matters. Apple's principle: the score is visible but not intrusive (a small badge, not a dashboard).

9. Organizational Timeline — From Log to Story

The Organizational Timeline (Daily Work #1) is a paginated, filterable chronological view of all signals across all providers. This is functional — the customer can see what happened. But a log is not a story. The customer has to interpret the events themselves.

Recommendation 9.1: Narrative Timeline

Current state: The timeline shows raw signal entries: 'pr.opened by priya.m on payments PR-447', 'slack.decision by raj.k', 'issue.transitioned by qa.lead'.

Deepened state: The timeline groups related signals into narrative events: 'Priya opened a PR for the payments circuit breaker. Raj reviewed it within 2 hours. QA blocked it pending compliance review. The PR was merged 3 days later after Legal signed off.' The customer reads a story, not a log. This is the Apple principle: the content is the interface. The raw signals are available on click (depth), but the default view is the narrative. Amazon's principle: the customer wants to understand what happened, not to decode event codes.

Implementation: Group signals by artifact (PR, ticket, doc) and time window (within 24 hours of each other). For each group, generate a one-sentence summary using the signal types and actors. This is rule-based for the pilot (template per artifact type: PR = opened/reviewed/merged/blocked, ticket = created/transitioned/resolved). In production, the LLM synthesizes the narrative from the signal list. The timeline shows narratives by default; clicking a narrative expands to show the raw signals.

10. Verified Knowledge — From Field to Workflow

The Verified Knowledge Layer (Competitor Feature C) added `verified_by` and `verified_at` fields to `OrganizationalLaw`. The `POST /laws/{code}/verify` endpoint lets a human sign off. This is the Guru lesson: verified knowledge is the differentiator. But verification currently requires the customer to navigate to the law and click 'Verify'. It is a manual, explicit step that most customers will never do.

Recommendation 10.1: Verification Prompts in Context

Current state: Laws have `verified_by`/`verified_at` fields. The customer must find the law and verify it manually.

Deepened state: When a law is cited in a briefing, explanation, or playbook output, it shows a verification badge: green checkmark (verified by a human), amber dot (auto-validated by the system but not human-verified), or red dot (candidate, not yet validated). Next to the amber dot, a 'Verify this' button appears. The customer verifies the law in the context where it is cited, not in a separate audit page. This is the Guru model: verification happens in the workflow, not in a separate tool. Apple's principle: the action is visible at the moment of relevance. Amazon's principle: reduce the distance between seeing and acting to zero.

Recommendation 10.2: Verification Expiry

Current state: Once verified, a law stays verified forever.

Deepened state: Verifications expire after 90 days (the Guru model: re-approve every 90 days). 7 days before expiry, the verification badge turns amber with a 'Re-verify' button. If the law has `drift_detected=True` (counter-evidence found since verification), the badge turns red automatically. This ensures verified knowledge stays current. The customer trusts Maestro's citations because they know every verified law was checked recently, not years ago. This is the Amazon principle: earn trust by being vocally self-critical. Maestro tells you when its knowledge might be stale.

11. Task Intelligence — From Extraction to Completion

The TaskExtractor (Daily Work #2) scans signal text during ingestion for action-item patterns and creates task learning objects. Tasks are filterable by assignee, domain, priority, status. This is functional. But tasks currently have only two states: open and done. There is no notion of progress, dependencies, or completion signals.

Recommendation 11.1: Auto-Completion Detection

Current state: Tasks stay 'open' forever unless manually marked done. The system does not detect when a task is completed.

Deepened state: When a new signal arrives that matches an open task (same artifact, same actor, completion-type signal like pr.merged or issue.transitioned to 'done'), Maestro automatically marks the task as 'kept' (completed). The task moves from 'Your tasks' to 'Completed this week' in TODAY. The customer sees Maestro closing loops without being asked. This is the Amazon flywheel: every completed task is a signal that feeds the learning loop. The customer feels Maestro tracking their work without creating a tracking system. Apple's principle: the system does the work invisibly.

Recommendation 11.2: Task Dependencies

Current state: Tasks are independent. No dependencies are tracked.

Deepened state: Maestro detects task dependencies from signal text: 'after the RFC is approved', 'once Legal signs off', 'pending the design review'. Tasks with dependencies show a 'Blocked by: {task}' label. The briefing surfaces blocked tasks: '3 tasks are blocked by the Legal review. Nudge legal@acme.com?' This makes the task list not just a to-do list, but a dependency graph. The customer sees not just what to do, but what is blocking what. This is the organizational physics (V3) applied to tasks: every task exists in a web of dependencies, and the critical path is the path with the most blocked tasks.

12. Conversational Curiosity — From Q&A; to Teaching

The Conversational Curiosity engine (V8 #3) asks the customer a question, takes their answer, asks a context-aware follow-up, and after 3 turns creates a `human_context` signal. This is the right pattern: the org teaches Maestro through conversation. But the conversation currently happens only in TODAY, and only when Maestro initiates it. The customer cannot start a conversation.

Recommendation 12.1: Customer-Initiated Teaching

Current state: Maestro asks questions in TODAY. The customer answers. The conversation is one-directional: Maestro initiates, the customer responds.

Deepened state: The customer can teach Maestro at any time. In the ASK surface, a 'Tell Maestro something' button opens a free-text input. The customer types: 'Legal always slows down OAuth approvals because Sarah needs to review every scope change.' Maestro processes this as a `human_context` signal, extracts the pattern (Legal + OAuth + Sarah + scope change = bottleneck), and confirms: 'Got it. I'll watch for this pattern in future OAuth PRs. Is this still true?' The customer teaches; Maestro learns; the model improves. This is the Amazon principle: the customer is the best source of truth. Maestro should make it effortless to inject human knowledge into the model. Apple's principle: the input is natural language, not a form.

Implementation: Add a POST `/api/oem/teach` endpoint that accepts free text, extracts entities + patterns using the existing `TaskExtractor` regex patterns (extended for knowledge statements), creates a `human_context` signal, and returns a confirmation that shows what Maestro learned. The confirmation is editable: 'I learned: Legal slows down OAuth because Sarah reviews scope changes. Is this correct?' The customer confirms or corrects. The corrected version is the final signal.

13. Cross-Feature Synergy — The Living Product

The design principles document states: 'Every daily action feeds the Yoda layer, and every Yoda insight improves the daily experience. The customer never sees the seams.' This is the vision. The current implementation has the pieces but not the wiring. The following synergies are the highest-leverage connections to build next.

13.1 The Morning Flywheel

The morning briefing (Daily Work #3) should be the hub of the flywheel. Every morning, the CEO opens the briefing and finds: today's top recommendation (from decision.py), drafted with a 'Why?' explanation (V8 #1) and a 'Create Jira ticket' button (Daily Work #4). Below it: commitments due today (Competitor Feature E), tasks extracted from yesterday's Slack messages (Daily Work #2), and any emerging unknowns detected overnight (V8 #2). The CEO reads, taps, and is done in 5 minutes. Every tap is a signal that makes tomorrow's briefing better. This is the Amazon flywheel: the briefing generates data, the data improves the model, the model improves the briefing.

13.2 The Trust Loop

Every insight Maestro shows has a verification badge (Competitor Feature C). Every explanation has a 'Why?' link (V8 #1). Every write-back has a preview (Daily Work #4). Every unknown is named honestly (V8 #2). Together, these create a trust loop: the customer sees evidence for every claim, can verify any law, can reject any action, and knows what Maestro does not know. This is the Guru lesson applied across the entire product: verified knowledge is the differentiator. The customer trusts Maestro because Maestro never bluffs. Over 90 days, this trust becomes the moat.

13.3 The Compounding Engine

Every feature in this document generates signals. The briefing generates attention signals (which items the CEO clicks). The write-back generates action signals (which tickets were created). The playbooks generate edit signals (how the customer modified the draft). The curiosity conversations generate human_context signals. The commitment tracker generates trust signals (kept vs broken). These signals feed back into the model, making every feature smarter. This is the Amazon principle: the flywheel accelerates itself. After 90 days of pilot data, the product should feel qualitatively different from day 1 — not because new features were added, but because the existing features learned from 90 days of real usage.

14. Priority Matrix — What to Build Next

The recommendations in this document range from 1-day tweaks to 2-week deep builds. The following matrix prioritizes them by customer impact (how much the customer notices) and implementation effort (how long it takes). Build the high-impact, low-effort items first.

Recommendation	Impact	Effort	Priority
2.1 Push delivery (briefing finds the CEO)	High	Low (1-2 days)	P0
2.2 One-tap execution (write-back in briefing)	High	Low (1 day)	P0
4.1 Synthesized answers (not result lists)	High	Medium (3-5 days)	P0
5.1 Inline explanations everywhere	High	Low (2 days)	P0
8.1 Commitments in the briefing	High	Low (1 day)	P0
10.1 Verification prompts in context	Medium	Low (1-2 days)	P1
3.1 Progressive trust (auto-execute)	High	Medium (3-5 days)	P1
7.1 Unknown to action pipeline	Medium	Medium (3 days)	P1
11.1 Auto-completion detection	Medium	Low (2 days)	P1
2.3 The briefing learns (attention signals)	High	Medium (5 days)	P1
6.1 Persona-adaptive drafts	High	High (7-10 days)	P2
9.1 Narrative timeline	Medium	Medium (5 days)	P2
12.1 Customer-initiated teaching	Medium	Medium (5 days)	P2
3.2 Contextual write-back suggestions	Medium	Medium (3-5 days)	P2
4.2 Follow-up questions	Medium	Low (2 days)	P2
6.2 Playbook learns from edits	Medium	High (7 days)	P3
8.2 Commitment health score	Low	Medium (3 days)	P3
10.2 Verification expiry (90 days)	Low	Low (1 day)	P3
11.2 Task dependencies	Low	Medium (3-5 days)	P3
5.2 Explanation as trust signal	Low	High (5-7 days)	P3

The P0 items (5 recommendations, ~7 days total) transform the product from functional to addictive. They are the difference between a tool the customer uses because they have to and a tool the customer opens because they want to. Build these first, in parallel with the 90-day pilot. By pilot day 30, the product should feel qualitatively different.

15. Conclusion

The 11 features built in this session are functional, tested, and wired. They represent a genuine cognitive layer for organizations — 19 invisible engines, 6 daily-work features, and 5 competitor-analysis features, all behind a 4-item sidebar. This is a strong foundation. But foundation is not product.

The recommendations in this document share one thread: they shift each feature from system-centric to customer-centric. The briefing finds the CEO instead of waiting to be found. The write-back executes with one tap instead of five. The explanation appears inline instead of requiring navigation. The verification happens in context instead of in an audit page. The task completes itself instead of waiting for manual closure. Every recommendation reduces the distance between insight and action. Every recommendation makes the system recede and the content emerge.

This is the Apple principle: the best interface is no interface. The best tool is the one that does its job so well that the customer forgets it exists. This is the Amazon principle: start with the customer and work backwards. The customer does not want a cognitive layer. They want their work done. They want their questions answered. They want their commitments tracked. They want their mornings organized. Maestro's job is to do all of this so effortlessly that the customer attributes the improvement to themselves, not to the tool.

Build the P0 recommendations. Ship the pilot. Let real organizations teach the system. Then watch the flywheel accelerate. The org becomes more capable, not more dependent. That is the moat. That is the product.